

Phase 1 - Week 2 - Lesson Day 3

Data Cleaning

Today's Objectives

Real-world data is almost never clean. Missing values, duplicate rows, and wrong data types are the norm, not the exception - and every analysis or model you build later is only as reliable as the cleaning work done here. This is arguably the single most important practical skill in data science, since messy data breaks everything downstream.

- Detect missing values in a dataset
- Handle missing values using `dropna()` and `fillna()`
- Detect and remove duplicate rows
- Convert columns to the correct data type

1. Finding Missing Values

Definition: Missing values in Pandas usually appear as **NaN** (Not a Number) - a special placeholder meaning "no value was recorded here." Detecting exactly where and how much data is missing is always the first step before deciding how to handle it.

```
df.isnull()           # True/False for every single cell - True means missing
df.isnull().sum()     # count of missing values PER COLUMN - the most useful version
df.isnull().sum().sum() # total missing values across the entire DataFrame
```

Why .sum() twice: the first `.sum()` adds up the True/False values (True counts as 1) down each column, giving you a per-column count. The second `.sum()` then adds those column totals together into one overall number.

2. Handling Missing Values

Definition: Once you know where data is missing, you generally choose one of two strategies: **drop** the affected rows entirely, or **fill** them with a reasonable placeholder value.

```
df.dropna()           # drop ANY row that has at least one missing value
df.dropna(subset=["age"]) # drop rows only if 'age' specifically is missing

df.fillna(0)          # replace ALL missing values with 0
df["age"].fillna(df["age"].mean()) # replace missing ages with the average age
df["city"].fillna("Unknown") # replace missing text values with a placeholder
```

Which one should you use? Use `dropna()` when missing data is rare and losing a few rows won't meaningfully affect your analysis. Use `fillna()` when you want to keep the row but need a sensible placeholder - filling numeric columns with the mean or median is extremely common, and filling text columns with a label like "Unknown" is standard practice.

3. Handling Duplicates

Definition: A *duplicate row* is a row that is an exact copy of another row already in the dataset. Duplicates often creep in from data entry errors, merging multiple data sources, or double-counted records, and they can quietly skew your statistics if left unchecked (for example, inflating a sum or average).

```
df.duplicated()          # True/False per row - True means it's a duplicate of an earlier row
df.duplicated().sum()    # count of duplicate rows
df.drop_duplicates()     # remove duplicate rows, keeping the FIRST occurrence of each
```

You can also check for duplicates based on specific columns only, using

df.duplicated(subset=["column_name"]) - useful when two rows might differ slightly but represent the same real-world record (for example, the same customer ID appearing twice).

4. Data Type Conversion

Definition: Sometimes a column that should be numeric gets loaded as text - often because of a stray symbol like a comma or a currency sign - or a date gets loaded as plain text instead of an actual date type. You cannot do math, filtering, or date-based operations correctly until the column has the right type.

```
df["age"] = df["age"].astype(int)          # convert a column to integer
df["price"] = df["price"].astype(float)    # convert a column to float
df["date"] = pd.to_datetime(df["date"])    # convert a column to an actual date type
```

A quick way to check if a conversion is needed: run **df.info()** and look at the **Dtype** column. If a column that should clearly be numeric shows up as **object** (Pandas' term for text), it needs converting before you can use it in calculations.

Today's Exercise

For this exercise, deliberately work with a "dirty" dataset - if your Day 1 dataset happens to be very clean already, search Kaggle for a dataset with the word "messy" or "dirty" in it (there are several specifically made for practicing cleaning), or manually introduce a few missing values and duplicate rows into your existing dataset for practice.

Exercise Questions (9 total)

1. Detect missing values

Run `.isnull().sum()` on your dataset and identify which columns have missing values and how many.

2. Total missing count

Print the total number of missing values across the entire dataset using `.isnull().sum().sum()`.

3. Drop rows with any missing value

Create a cleaned copy of your dataset using `dropna()`, and print how many rows were removed compared to the original.

4. Drop based on a specific column

Instead of dropping on any missing value, drop rows only where one specific important column is missing, using the `subset` parameter.

5. Fill numeric column with the mean

Pick a numeric column with missing values and fill them using that column's mean instead of dropping the rows.

6. Fill text column with a placeholder

Pick a text/categorical column with missing values and fill them with a placeholder like 'Unknown'.

7. Detect duplicates

Check how many duplicate rows exist in your dataset using `.duplicated().sum()`.

8. Remove duplicates

Remove the duplicate rows using `drop_duplicates()` and confirm the new row count is correct.

9. Fix a data type

Find a column that has the wrong data type (check with `.info()`), convert it to the correct type using `.astype()` or `pd.to_datetime()`, and verify the fix by running `.info()` again.

How to Approach It

Always check `.info()` and `.isnull().sum()` before AND after each cleaning step - this confirms your fix actually worked, and is a habit that will save you from silently broken data pipelines later in your career. Never assume a cleaning step worked just because it ran without an error.

Once you and Halimo have completed the exercises, bring your answers back to the lesson chat and we'll review them before moving to Lesson Day 4 (GroupBy & Merging).